

Keras 딥러닝 실습 정리

keras01 ~ keras25 요약 노트

NSU_ALL 레포지토리 | 2026-07-22

차례

1	딥러닝 기본 구조	keras01 ~ 06
2	MLP — 다중 입력·출력	keras07
3	훈련/테스트 데이터 분할	keras08
4	시각화와 회귀 평가지표	keras09 ~ 10
5	실전 데이터셋 회귀	keras11
6	학습 과정 제어	keras12 ~ 15
7	분류 모델 (이진 / 다중)	keras16 ~ 18, 20
8	GPU 환경 확인	keras19
9	모델 저장과 불러오기	keras21 ~ 23
10	과적합 방지와 데이터 전처리	keras24 ~ 25

1. 딥러닝 기본 구조

파일: keras01 ~ keras06

모델 만들기 — Sequential + Dense (keras01~03)

딥러닝 모델의 뼈대. Sequential은 층을 순서대로 쌓는 그릇이고, Dense는 완전연결층이다. 층을 여러 개 쌓으면 '딥'러닝이 된다. 첫 층에만 입력 크기(input_dim)를 알려주면 된다.

```
model = Sequential()
model.add(Dense(8, input_dim=1)) # 첫 층: 입력 1개 → 출력 8개
model.add(Dense(10))           # 은닉층
model.add(Dense(1))             # 출력층
```

컴파일과 훈련 — compile / fit

compile에서 loss(얼마나 틀렸는지 재는 기준)와 optimizer(틀린 것을 고치는 방법)를 정한다. 회귀에서는 loss='mse'(평균제곱오차), optimizer는 'adam'이 기본. fit의 epochs는 전체 데이터를 몇 번 반복 학습할지.

```
model.compile(loss='mse', optimizer='adam')
model.fit(x, y, epochs=100)
```

TIP epochs를 늘리면 loss가 줄어들지만, 무조건 많다고 좋은 건 아니다 (과적합 위험 → 6장 EarlyStopping 참고).

평가와 예측 — evaluate / predict (keras04)

evaluate는 학습이 끝난 모델의 loss를 계산하고, predict는 새 입력에 대한 예측값을 돌려준다.

```
loss = model.evaluate(x, y)
result = model.predict(np.array([7]))
```

batch_size (keras05)

데이터가 아주 많으면 한 번에 다 계산하지 않고 그룹으로 잘라서 계산한다. 이 그룹 크기가 batch_size. 작을수록 자주 업데이트되지만 느리고, 클수록 빠르지만 메모리를 많이 쓴다.

```
model.fit(x, y, epochs=100, batch_size=2)
```

행렬(다차원) 데이터 (keras06)

실제 데이터는 (행, 열) 형태의 행렬이다. 행 = 데이터 개수, 열 = 특성(컬럼) 개수. shape를 항상 확인하는 습관이 중요하다.

```
print(x.shape, y.shape) # 예: (442, 10) (442,)
```

2. MLP — 다중 입력·출력

파일: keras07_mlp1 ~ mlp4

입력 컬럼이 여러 개일 때 — input_shape

입력 특성이 2개 이상이면 input_dim 대신 input_shape=(열개수,)를 주로 쓴다. 이미지처럼 다차원 입력도 표현할 수 있기 때문 (예: input_shape=(28,28,1)). (None, 3)의 None은 '행(데이터 개수)은 몇 개든 상관없다'는 뜻.

```
model.add(Dense(12, input_shape=(3,))) # 컬럼 3개짜리 입력
```

mlp1 → mlp4 흐름

mlp1: 입력 컬럼 2개 / mlp2: 입력 컬럼 3개 / mlp3: 출력 여러 개 / mlp4: 다중입력+다중출력 종합. 핵심은 데이터 shape에 맞춰 첫 층의 input_shape와 마지막 층의 노드 수를 맞추는 것.

TIP x가 (10, 3)이면 input_shape=(3,), y가 (10, 2)면 마지막 Dense(2). 행이 아니라 열을 맞춘다!

3. 훈련/테스트 데이터 분할

파일: keras08_train_test1 ~ 3

왜 나누나?

학습에 쓴 데이터로 평가하면 '문제집을 외운 학생'처럼 성적이 부풀려진다. 그래서 훈련용(train)과 평가용(test)을 분리하고, 평가는 모델이 처음 보는 test로만 한다.

방법 1 — 넘파이 슬라이싱 (keras08_2)

리스트를 직접 잘라서 7:3으로 나누는 수동 방식.

```
x_train = x[0:7] # 앞 70%
x_test = x[7:10] # 뒤 30%
```

방법 2 — train_test_split (keras08_1, 08_3) ★실무 표준

sklearn 함수로 자동 분할. shuffle로 섞고, random_state를 고정하면 매번 같은 결과가 나와 실험 비교가 가능하다.

```
x_train, x_test, y_train, y_test = train_test_split(
    x, y, train_size=0.7, shuffle=True, random_state=368)
```

TIP random_state는 아무 숫자나 상관없다. '고정한다'는 것 자체가 중요.

4. 시각화와 회귀 평가지표

파일: keras09_scatter1~2, keras10_R2_RMSE

산점도로 결과 확인 (keras09)

matplotlib의 scatter로 실제값을 점으로 찍고, 예측 결과를 선으로 겹쳐 그리면 모델이 잘 맞추는지 눈으로 확인할 수 있다.

```
import matplotlib.pyplot as plt
plt.scatter(x, y) # 실제 데이터
plt.plot(x, results, color='red') # 모델 예측선
plt.show()
```

R2 스코어 — 결정계수 (keras10)

회귀 모델의 '설명력'. 1에 가까울수록 좋다. loss(mse)는 값 크기에 따라 커지므로, 0~1 스케일인 R2로 보면 직관적이다.

```
from sklearn.metrics import r2_score
r2 = r2_score(y_test, y_predict) # 원값, 예측값 순서
```

RMSE — 평균 제곱근 오차

mse는 오차를 제곱해서 값이 너무 커지므로, 루트를 씌운 RMSE를 함께 본다. 원래 데이터와 단위가 같아져 해석이 쉽다.

```
from sklearn.metrics import root_mean_squared_error
rmse = root_mean_squared_error(y_test, y_predict)
```

5. 실전 데이터셋 회귀

파일: keras11_1 ~ 11_3

캘리포니아 집값 (keras11_1)

fetch_california_housing() — 특성 8개, input_shape=(8,). 보스턴 집값 데이터가 인종 이슈로 제외되어 그 대안으로 쓰이는 표준 회귀 데이터셋.

```
datasets = fetch_california_housing()
x = datasets.data; y = datasets.target
```

당뇨 수치 (keras11_2)

load_diabetes() — 특성 10개, 442행. 교육용으로 자주 쓰는 작은 회귀 데이터셋.

캐글 자전거 수요예측 (keras11_3) — CSV 실전 흐름

처음으로 외부 CSV 파일을 pandas로 읽어 학습하고, 예측 결과를 제출 양식에 담아 CSV로 저장하는 실전 파이프라인.

```
train_csv = pd.read_csv('./_data/train.csv', index_col=0)
test_csv = pd.read_csv('./_data/test.csv', index_col=0)
# ... 학습 후 ...
submission.to_csv('./_data/sunmission_0717_1453.csv')
```

TIP index_col=0: 0번째 컬럼(날짜)은 데이터가 아니라 인덱스로 취급한다.

6. 학습 과정 제어

파일: keras12 ~ keras15

verbose — 로그 출력 조절 (keras12)

fit의 verbose 옵션으로 학습 중 출력을 조절한다. 0=조용히, 1=진행바(기본), 2=에포크당 한 줄. 출력을 끄면 학습 속도도 약간 빨라진다.

```
model.fit(x_train, y_train, epochs=100, verbose=0)
```

validation_split — 검증 데이터 (keras13)

훈련 데이터의 일부를 떼어 매 에포크마다 검증한다. 훈련 loss는 계속 줄어도 val_loss가 올라가기 시작하면 과적합 신호. 이 파일에서 activation='relu'(음수 제거)도 처음 등장한다. 기본값은 'linear'.

```
model.fit(x_train, y_train, epochs=100,
          validation_split=0.2) # 훈련의 20%를 검증에 사용
```

EarlyStopping — 조기 종료 (keras14)

val_loss가 patience 에포크 동안 개선되지 않으면 학습을 자동으로 멈춘다. restore_best_weights=True면 멈춘 시점이 아니라 가장 좋았던 시점의 가중치로 되돌린다.

```
from keras.callbacks import EarlyStopping
es = EarlyStopping(monitor='val_loss', mode='min',
                   patience=100, restore_best_weights=True)
model.fit(..., validation_split=0.2, callbacks=[es])
```

TIP mode='auto'로 두면 monitor 지표에 맞게 알아서 min/max를 정해준다. epochs는 크게 줘도 ES가 알아서 끊는다.

warnings — 경고 숨기기 (keras15)

실행 결과를 깔끔하게 보고 싶을 때 경고 메시지를 숨긴다.

```
import warnings
warnings.filterwarnings('ignore')
```

7. 분류 모델 (이진 / 다중)

파일: keras16 ~ 18, keras20

이진분류 — sigmoid + binary_crossentropy (keras16, 유방암)

정답이 0/1 두 가지면 이진분류. 마지막 층은 Dense(1, activation='sigmoid')로 0~1 사이 확률을 출력하고, loss는 반드시 binary_crossentropy. relu는 값을 0 이상으로만 보내므로 마지막 층에 쓰면 안 된다.

```
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam',
              metrics=['accuracy'])
# 예측값은 0~1 확률 → np.round()로 0/1 변환 후 accuracy_score
```

다중분류 — softmax + categorical_crossentropy (keras17 붓꽃, keras18 와인)

정답이 3개 이상이면 다중분류. 마지막 층은 클래스 수만큼 노드 + softmax(전체 합=1인 확률), loss는 categorical_crossentropy. y는 원핫인코딩으로 변환해야 한다 (예: 2 → [0,0,1]).

```
y = pd.get_dummies(y)      # 원핫인코딩
model.add(Dense(3, activation='softmax')) # 클래스 3개
model.compile(loss='categorical_crossentropy',
              optimizer='adam', metrics=['accuracy'])
```

argmax로 예측 클래스 뽑기 (keras20, 손글씨 숫자)

softmax 출력은 확률 배열이므로, np.argmax로 가장 확률 높은 칸의 번호를 뽑아 실제 클래스와 비교한다. load_digits는 8x8 손글씨 숫자 → 특성 64개, 클래스 10개.

```
y_predict_class = np.argmax(y_predict, axis=1)
acc = accuracy_score(y_test_class, y_predict_class)
```

TIP 정리: 회귀=마지막 linear+mse / 이진분류=sigmoid+binary_crossentropy /
다중분류=softmax+categorical_crossentropy

8. GPU 환경 확인

파일: keras19_gpu.test01

GPU 인식 확인

텐서플로가 GPU를 잡았는지 확인하는 코드. GPU가 있으면 학습이 훨씬 빨라진다.

```
import tensorflow as tf
gpus = tf.config.experimental.list_physical_devices('GPU')
if gpus: print('GPU 돈다~')
else:    print('GPU 없다~')
```

TIP 윈도우 네이티브에서 GPU를 쓰려면 TF 2.10 이하 필요 (수업 환경: tf291gpu = TF 2.9.1). 최신 TF는 WSL2 필요.

9. 모델 저장과 불러오기

파일: keras21 ~ keras23

model.summary() — 구조 확인 (keras21)

층별 출력 shape와 파라미터 수를 표로 보여준다. 파라미터 수 = (입력노드+1) × 출력노드 — +1은 bias.

```
model.summary()
```

save / load_model (keras22_1 ~ 22_4)

model.save()로 모델을 파일로 저장하고, load_model()로 다시 불러온다. compile 전에 저장하면 구조만, fit 후 저장하면 가중치까지 담긴다. load_model은 모델의 메서드가 아니라 독립 함수임에 주의.

```
model.save('./_save/keras_22_3.save_model.keras')
# --- 다른 파일에서 ---
from tensorflow.keras.models import load_model
model = load_model('./_save/keras_22_3.save_model.keras')
```

TIP 경로에 한글 윈도우 역슬래시를 쓸 땐 r'C:\...' (raw string) 또는 슬래시(/) 사용. \U 등이 이스케이프로 해석돼 SyntaxError가 난다.

ModelCheckpoint — 최적 모델 자동 저장 (keras23)

학습 중 val_loss가 가장 좋았던 순간의 모델을 자동으로 파일에 저장한다. EarlyStopping과 함께 callbacks에 넣어 쓰는 것이 표준 패턴.

```
from keras.callbacks import ModelCheckpoint
mcp = ModelCheckpoint(monitor='val_loss', mode='auto',
                      save_best_only=True, filepath='./_save/keras23_mcp1.keras')
model.fit(..., callbacks=[es, mcp])
```

10. 과적합 방지와 데이터 전처리

파일: keras24 ~ keras25

Dropout (keras24)

훈련할 때마다 지정 비율만큼 노드를 무작위로 꺼서 특정 노드 의존을 막는 과적합 방지 기법. 평가/예측 시에는 자동으로 전부 켜진다.

```
from keras.layers import Dense, Dropout
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.3)) # 30% 무작위 차단
```

MinMaxScaler — 정규화 (keras25)

특성마다 값 범위가 다르면(예: 나이 0~100 vs 연봉 0~1억) 큰 값이 학습을 지배한다. MinMaxScaler는 모든 값을 0~1 사이로 변환. fit은 train에만 하고 test는 transform만 한다 — test 정보가 학습에 새어 들어가는 것(데이터 누수)을 막기 위해.

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
scaler.fit(x_train) # 기준은 train으로만
x_train = scaler.transform(x_train)
x_test = scaler.transform(x_test) # test는 변환만
```

TIP 순서 주의: train_test_split 먼저 → 그 다음 스케일링.